

Thiago de Oliveira Sopeña Alves

DREAM RENDER

Trabalho de Conclusão de Curso apresentado
como requisito para a integralização curricular
do curso Técnico em Informática para Internet
integrado ao Ensino Médio.

Orientador:

Prof. Rafael Betito

RESUMO

Este trabalho explora os fundamentos da Computação Gráfica através do desenvolvimento de um motor gráfico tridimensional construído a partir de seus princípios matemáticos elementares. O projeto foca na implementação dos algoritmos responsáveis pela formação de imagens digitais, abordando desde a definição geométrica de objetos no espaço até a sua projeção e visualização em tela. O objetivo é analisar a viabilidade e o desempenho de dispositivos móveis modernos ao executarem essas operações matemáticas sem utilizar bibliotecas ou dependências externas nem o processador gráfico específico.

Palavras-chave: computação gráfica; javascript; dispositivos móveis.

LISTA DE ILUSTRAÇÕES

Figura 1: Vetor V entre os pontos $P1$ e $P2$ _____	11
Figura 2: Matriz de 4 linhas e 4 colunas _____	13
Figura 3: Projeção tridimensional para bidimensional _____	16
Figura 4: Divisão de uma face retangular em duas triangulares _____	17
Figura 5: Coordenadas baricêntricas _____	18
Figura 6: Transformações afim _____	21
Figura 7: Um retângulo em formato OBJ _____	25
Figura 8: Diagrama de Casos de Uso do Dream Render _____	27
Figura 9: Diagrama de Classes do Dream Render _____	29
Figura 10: Código para o cálculo das coordenadas de tela _____	32
Figura 11: Código para a ordenação de profundidade e desenho _____	33
Figura 12: Código para desenho pixel a pixel _____	34
Figura 13: Código para desenho com transformação afim _____	35
Figura 14: Tela com menu _____	36
Figura 15: Tela de navegação _____	37
Figura 16: Diferença na sensação de perspectiva entre os métodos de aplicação de textura _____	40

LISTA DE TABELAS

Tabela 1: Desempenho do dispositivo Samsung Galaxy A04S	38
Tabela 2: Desempenho do dispositivo Motorola Moto G73	39
Tabela 3: Desempenho do dispositivo iPhone 12	39
Tabela 4: Desempenho do dispositivo ASUS VivoBook 15	39

LISTA DE ABREVIATURAS E SIGLAS

HTML	<i>Hyper Text Markup Language</i> (Hyper linguagem de marcação de texto)
CSS	<i>Cascading Style Sheets</i> (Folhas de estilo em cascata)
JS	<i>Javascript</i>
JSON	<i>Javascript Object Notation</i> (Notação de Objeto Javascript)
CPU	<i>Central Processing Unit</i> (unidade central de processamento)
GPU	<i>Graphics Processing Unit</i> (unidade de processamento Gráfico)
OBJ	<i>Object</i> (Objeto)
API	<i>Application Programming Interface</i> (Interface de programação de
Aplicação)	
FPS	<i>Frames per second</i> (Quadros por segundo)
GHz	<i>Gigahertz</i>

SUMÁRIO

1. INTRODUÇÃO	8
1.1 Motivação	9
1.2 Objetivo	9
2. REFERENCIAL TEÓRICO	10
2.1 Fundamentação Matemática e Algoritmos	10
2.1.1 Ponto	10
2.1.2 Vetor	10
2.1.3 Matriz	13
2.1.4 Matriz de Translação	14
2.1.5 Matriz de Escala	14
2.1.6 Matriz de Rotação	15
2.1.7 Matriz de Modelo	15
2.1.8 Matriz de Visualização	15
2.1.9 Matriz de Projeção	16
2.1.10 Matriz MVP	17
2.1.11 Face	17
2.1.12 Coordenadas Baricêntricas	18
2.1.13 Coordenadas de Textura	18
2.2 Estruturação de Mundo Tridimensional	19
2.2.1 Cena	19
2.2.2 Objeto	19
2.2.3 Textura	19
2.3 Aplicação de Textura	19
2.3.1 Transformação afim	20
2.3.2 Pixel a Pixel	23
3. MATERIAIS E MÉTODOS	24
3.1 HTML e JavaScript	24
3.2 Wavefront OBJ	24
4. DREAM RENDER	26
4.1 Análise e Projeto	26
4.1.1 Requisitos	26
4.1.2 Diagrama de Casos de Uso	27
4.1.3 Diagrama de Classes	28
4.2 Implementação	31
4.2.1 Cálculo das Coordenadas de Tela	31

4.2.2 Ordenação de Profundidade e Desenho	33
4.2.3 Desenho Pixel a Pixel	33
4.2.4 Desenho com Transformação Afim	34
5. RESULTADOS	36
5.1 Capturas de Tela	36
5.2 Comparativo de Desempenho	37
5.2.1 Impacto do Método de Aplicação de Textura	39
5.2.2 Impacto do Processador	40
6. CONSIDERAÇÕES FINAIS	41
7. REFERÊNCIAS	42
GLOSSÁRIO	43

1. INTRODUÇÃO

O espaço tridimensional (3D) envolve cálculos em três eixos: X (horizontal), Y (vertical) e Z (profundidade). O eixo X se estende da esquerda para direita, o Y de baixo para cima e o Z de dentro ou para fora da tela (GAMBETTA, 2021). As coordenadas (X, Y, Z) permitem definir a posição de um ponto no espaço. O segmento de reta entre dois pontos é uma aresta. Três ou mais arestas formam uma face. Normalmente à cada face também está associada uma imagem ou textura. O conjunto de pontos, arestas, faces e texturas constitui um objeto tridimensional. Vários objetos, fontes de iluminação e uma câmera constituem uma cena.

A movimentação da câmera pela cena é feita pela aplicação de transformações geométricas aos objetos. Algumas transformações são: translação, rotação, escalonamento e projeção. Translação é o deslocamento de um objeto por uma distância em uma direção. Rotação é a rotação de um objeto ao redor de um eixo por um ângulo. Escalonamento é a variação do tamanho de um objeto por um fator de escala. Projeção é a transformação das coordenadas tridimensionais de um objeto em coordenadas na tela do computador (GAMBETTA, 2021). Todas estas transformações podem ser representadas por operações matriciais. Uma matriz é uma estrutura matemática com valores organizados na forma de uma tabela, com linhas e colunas (BLUNDELL, 2008).

Objetos localizados dentro do campo de visão da câmera são desenhados na tela. As texturas de cada face aparente são pintadas *pixel a pixel* considerando as projeções dos pontos na tela. Devido a todos os cálculos necessários, o processo de aplicação das texturas é computacionalmente custoso. Quando possível, este processo é transferido da CPU para a GPU. A CPU, unidade central de processamento, é um processador que executa uma instrução sobre um dado em um ciclo. A GPU, unidade de processamento gráfico, por sua vez, é um processador que executa uma instrução sobre múltiplos dados em um ciclo. Por isso é melhor utilizar a GPU, pois ela processa grandes quantidades de dados em paralelo, acelerando o desenho.

Com base no exposto, este trabalho busca responder à questão: qual é o requisito mínimo de capacidade de processamento necessário para que uma CPU consiga

desenhar uma cena tridimensional em um dispositivo móvel sem recorrer a uma GPU dedicada?

Para isto, serão realizados testes em diversos dispositivos, desenhando cenas tridimensionais com diferentes complexidades e utilizando duas técnicas de aplicação de textura.

1.1 Motivação

A motivação para este trabalho surge da necessidade de desmistificar as bibliotecas gráficas modernas. Além do viés educacional, existe o desafio técnico de testar os limites do hardware móvel atual. É relevante para a área de Informática para Internet investigar se os processadores (CPUs) de celulares modernos evoluíram o suficiente para lidar com cargas de trabalho gráficas intensivas que, tradicionalmente, seriam delegadas a hardware dedicado (GPU).

1.2 Objetivo

O objetivo geral deste trabalho é verificar se é possível desenhar uma cena tridimensional em um dispositivo móvel apenas utilizando a CPU.

Objetivos específicos:

- Estudar os fundamentos matemáticos da computação gráfica.
- Desenvolver um motor gráfico utilizando JavaScript, HTML e CSS, sem uso de bibliotecas externas ou *frameworks*, capaz de carregar cenas e arquivos com extensão OBJ.
- Implementar as ferramentas de desenho gráfico: transformações matriciais, projeção na tela e aplicação de textura, manualmente.
- Realizar testes de desempenho em três modelos de celulares, comparando duas técnicas de aplicação de textura.
- Analisar os resultados com base no tempo de desenho por *frame*.

2. REFERENCIAL TEÓRICO

Este capítulo apresenta os fundamentos teóricos necessários para o desenvolvimento de um motor gráfico. Abordam-se os conceitos de computação gráfica, a matemática subjacente ao processamento de imagens tridimensionais e as tecnologias *web* utilizadas para a implementação de um motor gráfico.

2.1 Fundamentação Matemática e Algoritmos

O desenvolvimento de um motor gráfico baseia-se inteiramente na aplicação de conceitos de Álgebra Linear e Geometria Analítica. Todas as operações descritas abaixo foram implementadas sem a utilização de bibliotecas externas.

2.1.1 Ponto

Um ponto representa uma posição dentro do espaço descrito por um sistema de coordenadas. Cada coordenada do ponto é definida por um valor. O número de coordenadas determina a dimensão do ponto (GAMBETTA, 2021). Um ponto P com 10 em x , 5 em y e 4 em z é escrito como $P = (10, 5, 4)$.

2.1.2 Vetor

Um vetor é uma entidade que descreve a relação entre dois pontos. Essa relação é composta pela direção e comprimento e é normalmente visualizado por uma seta (GAMBETTA, 2021).

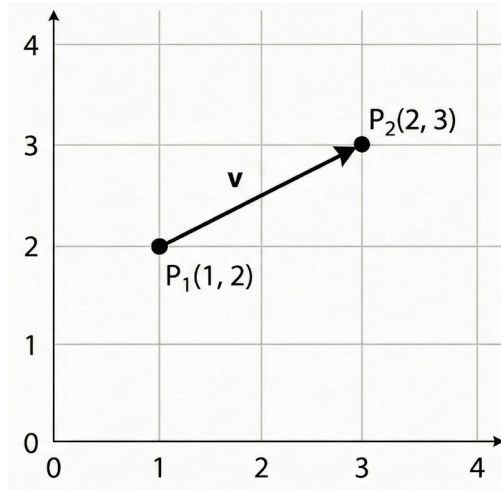


Figura 1 - Vetor V entre os pontos P1 e P2.

Na figura 1, o vetor $\vec{V}$ é o resultado da subtração $P_2 - P_1$, que pode ser obtida através de $(2, 3) - (1, 2) = (2 - 1, 3 - 2) = (1, 1)$. Obtendo assim $\vec{V} = (1, 1)$, com 1 em x e 1 em y.

Comprimento

O comprimento l de um vetor $\vec{V}$ é obtido pelo teorema de Pitágoras conforme a equação 1 (GAMBETTA, 2021).

$$l = |\vec{V}| = \sqrt{x_V^2 + y_V^2 + z_V^2} \quad (1)$$

Normalização

Um vetor normalizado é obtido dividindo um vetor $\vec{V}$ por seu comprimento l . conforme a equação 2. O comprimento do vetor normalizado é sempre igual a 1 (GAMBETTA, 2021).

$$\frac{\vec{V}}{l} = \left(\frac{x_V}{l}, \frac{y_V}{l}, \frac{z_V}{l} \right) \quad (2)$$

Adição e Subtração de Vetores

Considerando os vetores $\vec{A}$ e $\vec{B}$, a adição é definida pela equação 3 e a subtração, pela equação 4 (GAMBETTA, 2021)

$$\vec{A} + \vec{B} = (x_A + x_B, y_A + y_B, z_A + z_B) \quad (3)$$

$$\vec{A} - \vec{B} = (x_A - x_B, y_A - y_B, z_A - z_B) \quad (4)$$

Multiplicação e Divisão de Vetor por Escalar

Estas operações envolvem um vetor $\vec{V}$ e um número real n . Nestas operações, multiplicamos ou dividimos todos os elementos de $\vec{V}$ por n , conforme as equações 5 e 6. O resultado destas operações é um novo vetor com a mesma direção de $\vec{V}$, mas com seu comprimento alterado (GAMBETTA, 2021).

$$\vec{V}n = (x_V n, y_V n) \quad (5)$$

$$\frac{\vec{V}}{n} = \left(\frac{x_V}{n}, \frac{y_V}{n} \right) \quad (6)$$

Produto Escalar

O produto escalar é uma operação algébrica entre dois vetores que resulta em um número real, e não em um novo vetor (GAMBETTA, 2021). Considerando os vetores $\vec{A}$ e $\vec{B}$, o produto escalar é definido pela equação 7.

$$\vec{A} \cdot \vec{B} = x_A x_B + y_A y_B + z_A z_B = |\vec{A}| |\vec{B}| \cos(\alpha) \quad (7)$$

O produto escalar é relacionado ao cosseno do ângulo α entre os vetores. Se o ângulo α entre os vetores for 90° , o resultado será 0. Se α for 0° , o resultado será positivo. Se α for 180° , o resultado será negativo (GAMBETTA, 2021).

Produto Vetorial

Dados os vetores $\vec{A}$ e $\vec{B}$, o produto vetorial é uma operação que resulta em um terceiro vetor que é simultaneamente perpendicular a $\vec{A}$ e $\vec{B}$. Se $\vec{A}$ e $\vec{B}$ definem um plano, $\vec{A} \times \vec{B}$ resulta no vetor normal deste plano (GAMBETTA, 2021). O produto vetorial é definido pela equação 8.

$$\vec{A} \times \vec{B} = (y_A z_B - z_A y_B, z_A x_B - x_A z_B, x_A y_B - y_A x_B) \quad (8)$$

2.1.3 Matriz

Uma matriz é uma estrutura matemática fundamental composta por valores organizados em um arranjo retangular de linhas e colunas (GAMBETTA, 2021). A figura 2 mostra uma matriz de 4 linhas e 4 colunas, onde a primeira linha é composta por a , b , c e d e a primeira coluna, por a , e , i e m .

$$\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix}$$

Figura 2 - Matriz de 4 linhas e 4 colunas.

No contexto da computação gráfica, as matrizes servem como operadores que transformam o espaço geométrico, permitindo mover, girar e distorcer objetos (GAMBETTA, 2021).

Multiplicação de Matrizes

Sendo A uma matriz m por n e B uma matriz n por p , a multiplicação AB resulta em uma nova matriz C , m por p , onde o elemento de C na linha i e na coluna j é obtido através do produto escalar entre a linha i da matriz A e a coluna j da matriz B , conforme a equação 9 (GAMBETTA, 2021).

$$C = AB = \begin{bmatrix} A_{lin_1} \cdot B_{col_1} & A_{lin_1} \cdot B_{col_2} & \dots & A_{lin_1} \cdot B_{col_p} \\ A_{lin_2} \cdot B_{col_1} & A_{lin_2} \cdot B_{col_2} & \dots & A_{lin_2} \cdot B_{col_p} \\ \vdots & \vdots & \ddots & \vdots \\ A_{lin_m} \cdot B_{col_1} & A_{lin_m} \cdot B_{col_2} & \dots & A_{lin_m} \cdot B_{col_p} \end{bmatrix} \quad (9)$$

Multiplicação de matriz com Ponto/Vetor

A multiplicação de uma matriz M com um ponto/vetor $\vec{V}$ ocorre da mesma forma que a multiplicação entre matrizes. A diferença é que $M\vec{V}$ resulta em um novo ponto/vetor, conforme a equação 10 (GAMBETTA, 2021).

$$M\vec{V} = \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} (x, y, z, w) =$$

$$(xa + yb + zc + wd, xe + yf + zg + wh, xi + yj + zk + wl, xm + yn + zo + wp) \quad (10)$$

Coordenadas Homogêneas

As coordenadas homogêneas permitem utilizar a multiplicação matricial como forma de aplicar todas as transformações geométricas em pontos e vetores (GAMBETTA, 2021).

Em um motor gráfico tridimensional, todas as transformações geométricas são representadas por matrizes com 4 linhas e 4 colunas. Como só é possível multiplicar uma matriz com 4 linhas e 4 colunas por um vetor com 4 elementos, uma quarta dimensão w é adicionada aos vetores e aos pontos tridimensionais. Nos pontos, é utilizado 1 para w , enquanto que nos vetores, é utilizado 0 (GAMBETTA, 2021).

2.1.4 Matriz de Translação

A translação é uma transformação geométrica que desloca um objeto de uma posição para outra no espaço tridimensional. Ocorre quando se multiplica uma matriz de translação M_T por um ponto ou vetor em coordenadas homogêneas, conforme a equação 11. Os valores T_x , T_y e T_z , representam os valores de deslocamento linear para cada eixo (BLUNDELL, 2008).

$$M_T\vec{V} = \begin{bmatrix} 1 & 0 & 0 & T_x \\ 0 & 1 & 0 & T_y \\ 0 & 0 & 1 & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix} (x, y, z, w) = (x + T_x, y + T_y, z + T_z, w) \quad (11)$$

2.1.5 Matriz de Escala

O escalonamento é uma transformação geométrica que altera as dimensões de um objeto por um fator de escala. Ocorre quando se multiplica uma matriz de escalonamento

M_S por um ponto ou vetor em coordenadas homogêneas, conforme a equação 12. Os valores S_x , S_y e S_z , representam os fatores de escala para cada eixo (BLUNDELL, 2008).

$$M_S \vec{V} = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} (x, y, z, w) = (xS_x, yS_y, zS_z, w) \quad (12)$$

2.1.6 Matriz de Rotação

A rotação é uma transformação geométrica que altera o ângulo de um objeto em relação a um eixo. Ocorre quando se multiplica uma matriz de rotação M_R por um ponto ou vetor em coordenadas homogêneas. Na equação 13, (x, y, z) é o eixo sobre o qual o objeto será rotacionado e a o ângulo de rotação (BLUNDELL, 2008).

$$M_R = \begin{bmatrix} 1 + (1 - \cos a)(x^2 - 1) & z \sin a + xy(1 - \cos a) & -y \sin a + xz(1 - \cos a) & 0 \\ -z \sin a + xy(1 - \cos a) & 1 + (1 - \cos a)(y^2 - 1) & x \sin a + yz(1 - \cos a) & 0 \\ y \sin a + xz(1 - \cos a) & -x \sin a + yz(1 - \cos a) & 1 + (1 - \cos a)(z^2 - 1) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (13)$$

2.1.7 Matriz de Modelo

Os objetos tridimensionais armazenados em arquivos OBJ possuem pontos com coordenadas entre 0 e 1 (MURRAY; VAN RYPER, 1996). Para ajustar a posição, tamanho e orientação de cada objeto em uma cena, são realizadas as operações de translação, rotação e escalonamento. A matriz de modelo M_M é a matriz resultante da multiplicação das matrizes de translação, rotação e escalonamento utilizadas, conforme a equação 14 (BLUNDELL, 2008).

$$M_M = M_T \times M_R \times M_S \quad (14)$$

2.1.8 Matriz de Visualização

Uma câmera é definida por um ponto P , que indica sua localização tridimensional, e um vetor $\vec{V}$, que indica a direção de observação. $\vec{A}$, $\vec{B}$ e $\vec{C}$ são vetores auxiliares normalizados. A matriz de visualização M_V , conforme a equação 15, é utilizada para ajustar os objetos na cena de acordo com o ponto de vista da câmera (BLUNDELL, 2008).

$$\vec{C} = P - \vec{V}$$

$$\vec{A} = \vec{C} \times (0, 1, 0)$$

$$\vec{B} = \vec{A} \times \vec{C}$$

$$M_V = \begin{bmatrix} x_A & y_A & z_A & -(\vec{A} \cdot \vec{V}) \\ x_B & y_B & z_B & -(\vec{B} \cdot \vec{V}) \\ x_C & y_C & z_C & -(\vec{C} \cdot \vec{V}) \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (15)$$

2.1.9 Matriz de Projeção

A matriz de projeção é responsável pela etapa final para desenhar um objeto em tela: a transformação do espaço tridimensional para um plano bidimensional. Esta matriz introduz a distorção necessária para criar a ilusão de profundidade com base em um plano próximo (*near clipping plane*) e um plano distante (*far clipping plane*) (DE VRIES, 2020), como mostra a figura 3.

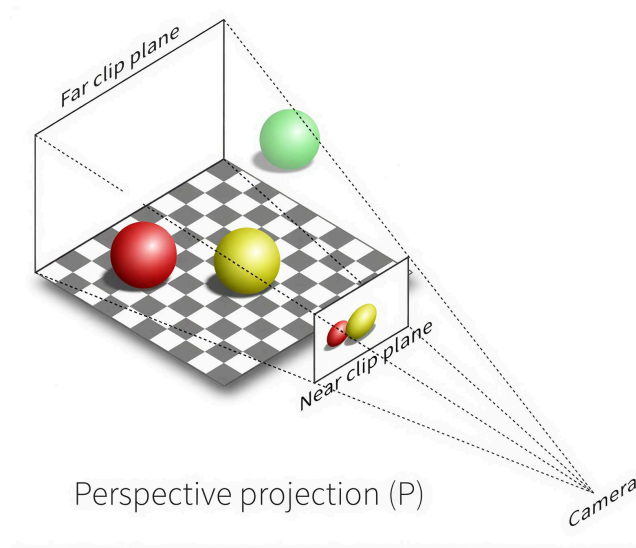


Figura 3 - Projeção tridimensional para bidimensional.

Fonte: (DE VRIES, 2020)

A matriz M_P , na equação 16, se baseia nos valores n e f , l e r , t e b que definem o volume de visualização. n é a distância até o plano próximo e f a distância até

o plano distante. (l, b) define o canto inferior esquerdo e (r, t) o canto superior direito do plano próximo (DE VRIES, 2020).

$$M_P = \begin{bmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad (16)$$

2.1.10 Matriz MVP

A matriz M_{MVP} é o resultado da multiplicação das matrizes M_M , M_V e M_P , conforme a equação 17 (BLUNDELL, 2008).

$$M_{MVP} = M_P \times M_V \times M_M \quad (17)$$

Multiplicando cada ponto (x, y, z, w) de um objeto na cena pela matriz M_{MVP} é obtido um novo ponto (x', y', z', w') dentro do volume de visualização. x' , y' e z' são números entre -1 e 1 (BLUNDELL, 2008).

Por fim, as equações 18 e 19 definem como são obtidas as coordenadas x_t e y_t em tela (BLUNDELL, 2008). l , r , t e b são os valores definidos na seção 2.1.9.

$$x_t = \frac{x' + 1}{2}(r - l) \quad (18)$$

$$y_t = \frac{1 - y'}{2}(b - t) \quad (19)$$

2.1.11 Face

O segmento de reta entre dois pontos é uma aresta. Três ou quatro arestas formam uma face. Faces formadas por quatro arestas são divididas em duas faces (GAMBETTA, 2021). A figura 4 mostra uma face retangular dividida em duas faces triangulares.

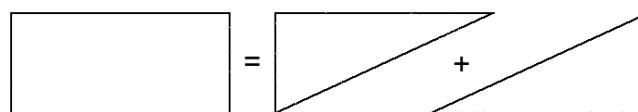


Figura 4 - Divisão de uma face retangular em duas triangulares.

2.1.12 Coordenadas Baricêntricas

As coordenadas baricêntricas permitem descrever qualquer ponto P no interior de uma face formada pelos pontos A , B e C , conforme a equação 20 e apresentado na figura 5. α , β e γ são números normalizados entre 0 e 1 de forma que $\alpha + \beta + \gamma = 1$ (BLUNDELL, 2008).

$$P = \alpha A + \beta B + \gamma C \quad (20)$$

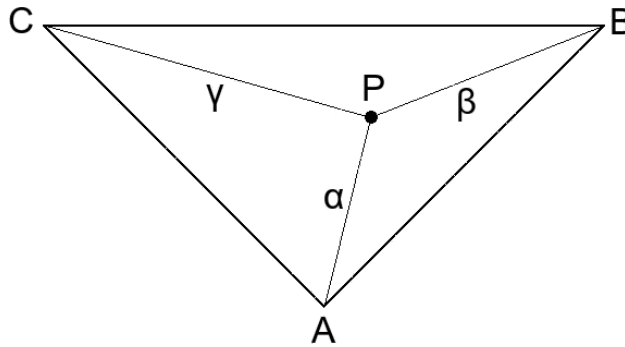


Figura 5 - Coordenadas baricêntricas.

Os valores de α , β e γ são calculados conforme as equações 21, 22 e 23 (BLUNDELL, 2008). A função area_{Δ} calcula a área de um triângulo formado por três pontos.

$$\alpha = \frac{\text{area}_{\Delta}(P, B, C)}{\text{area}_{\Delta}(A, B, C)} \quad (21)$$

$$\beta = \frac{\text{area}_{\Delta}(A, P, C)}{\text{area}_{\Delta}(A, B, C)} \quad (22)$$

$$\gamma = \frac{\text{area}_{\Delta}(A, B, P)}{\text{area}_{\Delta}(A, B, C)} \quad (23)$$

2.1.13 Coordenadas de Textura

Textura é uma imagem que será desenhada sobre uma face. Para se diferenciar das coordenadas em tela, as coordenadas em textura são denotadas por (u, v) , sendo u no eixo horizontal e v no vertical. Tanto u quanto v são números normalizados entre 0 e

1. O canto superior esquerdo da imagem possui $(u, v) = (0, 0)$ e o canto inferior direito $(u, v) = (1, 1)$ (GAMBETTA, 2021).

2.2 Estruturação de Mundo Tridimensional

Assim como na seção 2.1, todas as estruturas descritas abaixo foram implementadas sem a utilização de bibliotecas externas.

2.2.1 Cena

A cena no espaço tridimensional é uma coleção de objetos, fontes de iluminação e câmeras. De cada um destes são armazenados a posição, a orientação e o tamanho. Estes dados são utilizados pelo motor gráfico para desenhar a cena (GAMBETTA, 2021).

2.2.2 Objeto

Um objeto tridimensional é uma coleção de pontos ou vértices, arestas e faces. De cada face são armazenados os vértices e arestas que a compõe, o centro geométrico, o vetor normal e, opcionalmente, uma textura e o mapeamento das coordenadas de textura (GAMBETTA, 2021).

2.2.3 Textura

Textura é uma imagem bidimensional que será desenhada sobre uma face para criar a ilusão de objetos sólidos construídos com diferentes materiais (GAMBETTA, 2021).

2.3 Aplicação de Textura

Essa seção explica dois métodos de aplicação de textura utilizados por um motor gráfico, transformação afim e *pixel a pixel*.

2.3.1 Transformação afim

As transformações afim são transformações geométricas no espaço bidimensional que permitem translação, rotação, espelhamento, escalonamento e cisalhamento. A matriz de transformação M_{affine} é definida pela equação 24 (SALOMON, 2006).

$$M_{affine} = \begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix} \quad (24)$$

Multiplicando a matriz M_{affine} por cada ponto P de uma imagem, são obtidas novas coordenadas conforme a equação 25 (SALOMON, 2006).

$$M_{affine}P = \begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix} (x, y) = (ax + cy + e, bx + dy + f) \quad (25)$$

A figura 6 apresenta várias configurações da matriz, demonstrando os efeitos dos valores de a , b , c , d , e e f no resultado. Como pode ser observado, e e f controlam a translação, a e d o escalonamento, a , b , c , d a rotação, b e c o cisalhamento e a e d o espelhamento.

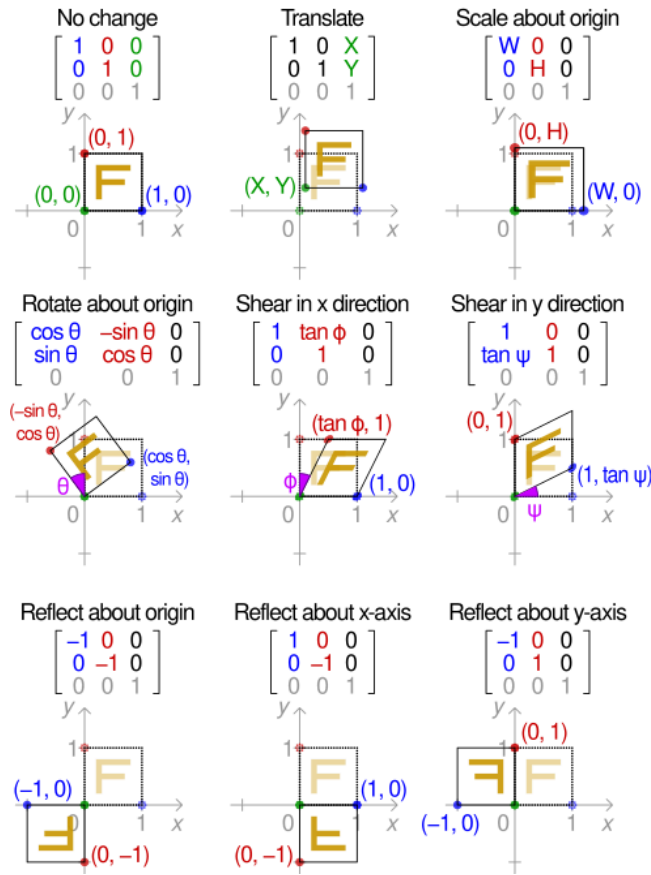


Figura 6 -Transformações afim.

Fonte: https://en.wikipedia.org/wiki/Affine_transformation

Para aplicar textura em uma face com transformação afim, primeiro são obtidas as coordenadas (x_t, y_t) dos vértices da face utilizando a matriz M_{MVP} e definido um triângulo na tela. Sabendo os pontos onde a textura deve ser desenhada na tela, é possível calcular os valores a , b , c , d , e e f da equação 25 de modo que satisfaçam os dois sistemas de equações 26.

$$\begin{cases} au_0 + bv_0 + c = x_0 \\ au_1 + bv_1 + c = x_1 \\ au_2 + bv_2 + c = x_2 \end{cases} \quad \begin{cases} du_0 + ev_0 + f = y_0 \\ du_1 + ev_1 + f = y_1 \\ du_2 + ev_2 + f = y_2 \end{cases} \quad (26)$$

Usando a regra de Cramer e o esquema de Sarrus para determinantes de matrizes três por três, δ pode ser definido pela equação 27, δ_a pela 28, δ_b pela 29, δ_c pela 30, δ_d pela 31, δ_e pela 32, δ_f pela 33. Por fim, os valores a , b , c ,

d , e e f podem ser obtido pelas equações 34 a 39 (IEZZI *et al.*, 2016). Por fim, a transformação é aplicada à textura e desenhada na tela.

$$\delta = \text{determinant} \left(\begin{bmatrix} u_0 & v_0 & 1 \\ u_1 & v_1 & 1 \\ u_2 & v_2 & 1 \end{bmatrix} \right) \quad (27)$$

$$\delta_a = \text{determinant} \left(\begin{bmatrix} x_0 & v_0 & 1 \\ x_1 & v_1 & 1 \\ x_2 & v_2 & 1 \end{bmatrix} \right) \quad (28)$$

$$\delta_b = \text{determinant} \left(\begin{bmatrix} u_0 & x_0 & 1 \\ u_1 & x_1 & 1 \\ u_2 & x_2 & 1 \end{bmatrix} \right) \quad (29)$$

$$\delta_c = \text{determinant} \left(\begin{bmatrix} u_0 & v_0 & x_0 \\ u_1 & v_1 & x_1 \\ u_2 & v_2 & x_2 \end{bmatrix} \right) \quad (30)$$

$$\delta_d = \text{determinant} \left(\begin{bmatrix} y_0 & v_0 & 1 \\ y_1 & v_1 & 1 \\ y_2 & v_2 & 1 \end{bmatrix} \right) \quad (31)$$

$$\delta_e = \text{determinant} \left(\begin{bmatrix} u_0 & y_0 & 1 \\ u_1 & y_1 & 1 \\ u_2 & y_2 & 1 \end{bmatrix} \right) \quad (32)$$

$$\delta_f = \text{determinant} \left(\begin{bmatrix} u_0 & v_0 & y_0 \\ u_1 & v_1 & y_1 \\ u_2 & v_2 & y_2 \end{bmatrix} \right) \quad (33)$$

$$a = \frac{\delta_a}{\delta} \quad (34)$$

$$b = \frac{\delta_b}{\delta} \quad (35)$$

$$c = \frac{\delta_c}{\delta} \quad (36)$$

$$d = \frac{\text{delta}_d}{\text{delta}} \quad (37)$$

$$e = \frac{\text{delta}_e}{\text{delta}} \quad (38)$$

$$f = \frac{\text{delta}_f}{\text{delta}} \quad (39)$$

2.3.2 Pixel a Pixel

Para desenhar a textura em uma face, primeiro são obtidas as coordenadas (x_t, y_t) dos vértices da face utilizando a matriz M_{MVP} e definido um triângulo na tela. Depois, para cada ponto (x, y) dentro do triângulo na tela são calculados os valores de α , β e γ das coordenadas baricêntricas. Em seguida, são calculados as coordenadas de textura (u, v) com base nos valores de α , β e γ . Por fim, o *pixel* nas coordenadas (u, v) na textura é obtido e desenhado nas coordenadas (x, y) na tela (GAMBETTA, 2021).

3. MATERIAIS E MÉTODOS

Este capítulo apresenta as tecnologias selecionadas para o desenvolvimento de um motor gráfico. A seleção priorizou a execução nativa em navegadores *web*, eliminando a necessidade de instalações de *software* adicionais nos dispositivos de teste.

3.1 HTML e JavaScript

HTML é uma linguagem de marcação de texto usada para definir a estrutura de páginas *web* (MOZILLA DEVELOPER NETWORK, 2025a).

JavaScript é uma linguagem de programação orientada a objetos interpretada que permite a manipulação dinâmica de páginas *web* (MOZILLA DEVELOPER NETWORK, 2025b).

O HTML e o JavaScript foram adotados devido ao suporte nativo em navegadores *web* e por possuírem `<canvas>` e Canvas API. O elemento `<canvas>` do HTML é uma superfície de desenho que suporta uma grande variedade de funções JavaScript da Canvas API (MOZILLA DEVELOPER NETWORK, 2025c).

3.2 Wavefront OBJ

Para a representação dos objetos tridimensionais, foi adotado o formato Wavefront OBJ. Este é um formato de arquivo aberto e baseado em texto, que lista vértices, faces, normais e coordenadas de textura, amplamente suportado pelas ferramentas de modelagem tridimensional atuais (MURRAY; VAN RYPER, 1996).

A figura 7 descreve um arquivo OBJ com um retângulo. A linha 2, com o identificador `o`, define o nome do objeto como `rectangle`. As linhas 3 a 6, com o identificador `v`, definem os vértices com os valores separados por espaço. O primeiro vértice, na linha 3, possui $(x, y, z) = (-1, -1, 1)$. As linhas 7 a 10, com o identificador `vt`, definem as coordenadas de textura no sentido horário, com os valores separados por espaço. A linha 11, com o identificador `vn`, define um vetor normal de face, com os valores separados por espaço. As linhas 13 e 14, com o identificador `f`, definem as faces

com conjuntos separados por espaço e valores separados por barras. Na linha 13, a face é definida por três conjuntos. O primeiro conjunto da face referencia o ponto 1 (linha 3), mapeia a coordenada de textura 1 (linha 7) e utiliza a normal 1 (linha 11). O segundo conjunto da face referencia o ponto 2 (linha 4), mapeia a coordenada de textura 2 (linha 8) e utiliza a normal 1 (linha 11). O terceiro conjunto da face referencia o ponto 3 (linha 5), mapeia a coordenada de textura 3 (linha 9) e utiliza a normal 1 (linha 11).

```
1  mllib Program/Graphics/Models/rectangle.mtl
2  o rectangle
3  v -1.0 -1.0 1.0
4  v 1.0 -1.0 1.0
5  v 1.0 1.0 1.0
6  v -1.0 1.0 1.0
7  vt 0.0 0.0
8  vt 1.0 0.0
9  vt 1.0 1.0
10 vt 0.0 1.0
11 vn 0.0 0.0 1.0
12 usemtl box
13 f 1/1/1 2/2/1 3/3/1
14 f 1/1/1 3/3/1 4/4/1
```

Figura 7 - Um retângulo em formato OBJ.

4. DREAM RENDER

Este capítulo apresenta aspectos de análise, projeto e implementação do motor gráfico Dream Render.

4.1 Análise e Projeto

Nesta seção, serão discutidos os requisitos, os casos de uso e as classes necessárias.

4.1.1 Requisitos

A etapa de elicitação de requisitos definiu as funcionalidades necessárias para desenhar e navegar em cenas tridimensionais sem dependência de bibliotecas externas.

Requisitos Funcionais

RF01 - Carregamento de cena

Prioridade: Alta

Descrição: O Dream Render deve permitir o carregamento de arquivos no formato OBJ que descrevem a cena, objetos e texturas a serem utilizadas.

Critérios de Aceite: Um objeto de cena tem que ser criado.

RF02 - Movimento de cena

Prioridade: Alta

Descrição: O Dream Render deve permitir que o usuário se movimente pela cena.

Critérios de Aceite: Os parâmetros da câmera devem ser alterados.

RF03 - Desenho em tela

Prioridade: Alta

Descrição: O Dream Render deve exibir somente as faces visíveis dos objetos que estão no campo de visão, aplicando suas texturas.

Critérios de Aceite: A cena deve ser exibida na tela.

RF04 - Mostrar desempenho

Prioridade: Alta

Descrição: O Dream Render deve exibir um contador de *frames* por segundo em tempo real.

Critérios de Aceite: O contador deve aparecer na tela.

Requisitos Não Funcionais

RNF01 - Compatibilidade

O Dream Render deve rodar em navegadores *web* com suporte a HTML Canvas.

RNF02 - Isolamento

O Dream Render deve processar os gráficos totalmente no navegador *web*.

RNF03 - Independência

O Dream Render não deve utilizar bibliotecas ou dependências externas.

4.1.2 Diagrama de Casos de Uso

O diagrama de casos de uso do Dream Render é apresentado na Figura 8. Apresenta o único ator: usuário e suas ações: carregar cena e desenhar cena. O caso de uso carregar cena carrega uma cena existente. O caso de uso desenhar cena, aplica as texturas nas faces visíveis de todos os objetos no campo de visão de câmera e permite a movimentação da câmera pela cena.

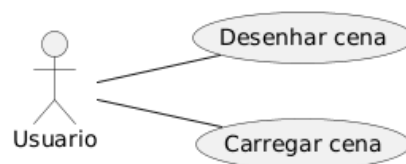


Figura 8 - Diagrama de Casos de Uso do Dream Render.

4.1.3 Diagrama de Classes

O diagrama de classes do Dream Render é apresentado na Figura 9. Apresenta as classes `Vec3D`, `Texture`, `Face`, `Mesh`, `Graphics`, `Transform`, `Camera`, `Component`, `Object` e `Scene`, detalhando seus atributos, métodos e os relacionamentos entre elas. A arquitetura adota um padrão baseado em composição de componentes, permitindo a criação de componentes personalizados sem a necessidade da modificação dos códigos existentes do motor gráfico.

Como foi escolhida a linguagem JavaScript para implementação do motor gráfico, duas observações são importantes sobre o diagrama de classes. A classe `Object` é do Dream Render, enquanto que a classe `object` é a classe genérica do JavaScript. E também, todos atributos das classes são públicos.

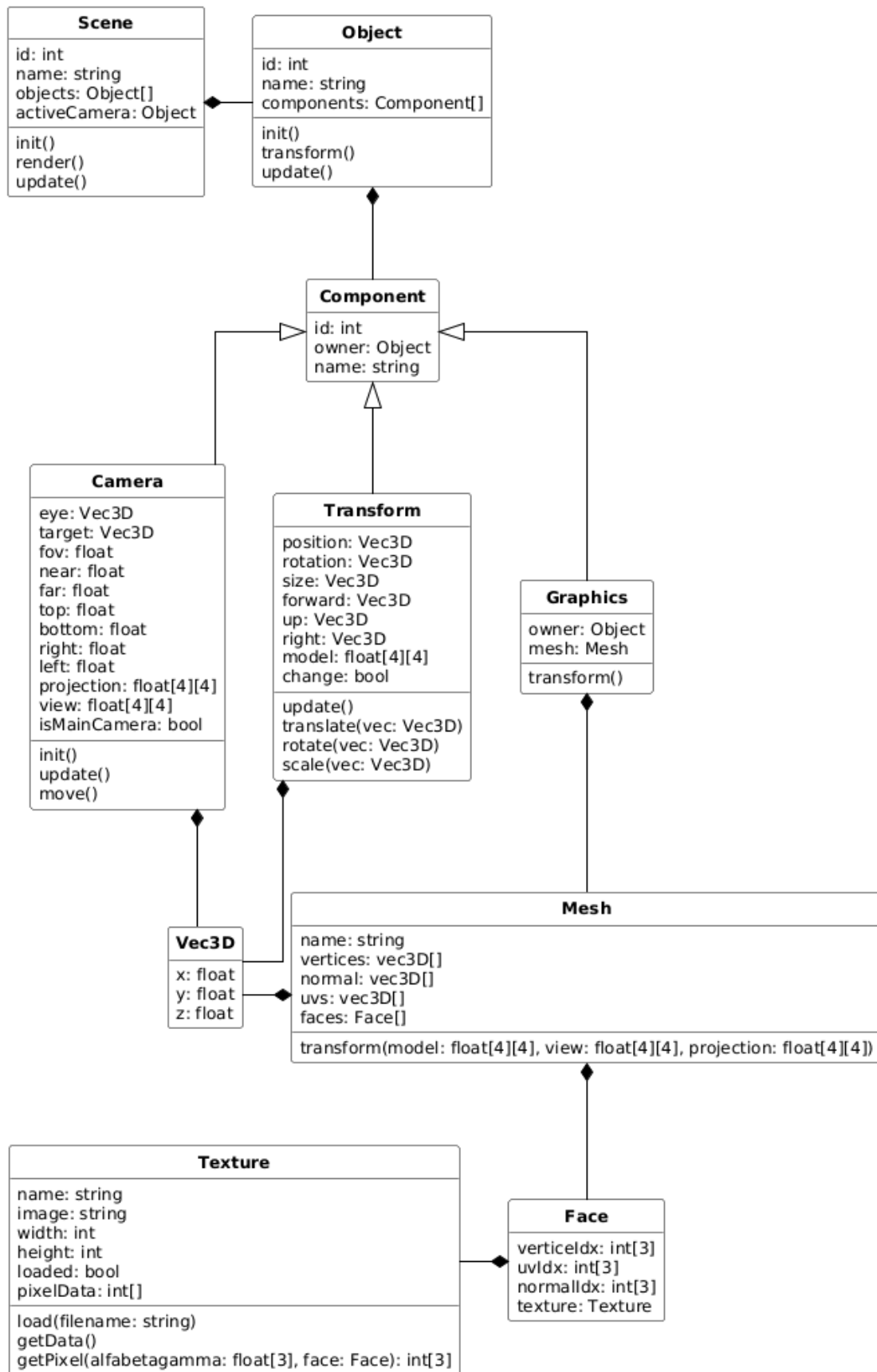


Figura 9 - Diagrama de Classes do Dream Render.

A classe `Vec3D` é utilizada para representar vetores, pontos e ângulos diretores tridimensionais.

A classe `Texture` é utilizada para armazenar informações dos *pixels* de uma imagem. O método `load()` carrega a imagem. O método `getPixel()` retorna um *pixel* da imagem utilizando os dados da *face* e o *alfabetagamma* para calcular as coordenadas baricêntricas.

A classe `Face` é utilizada para armazenar as informações das faces de um objeto. No Dream Render, todas as faces são triangulares e texturizadas. Para cada ponto do triângulo são armazenados o vértice, as coordenadas de textura uv e a normal.

A classe `Mesh` é utilizada para armazenar em memória um arquivo no formato Wavefront OBJ. O método `transform()` posiciona os objetos na cena e verifica as faces aparentes dentro do volume de visualização utilizando as matrizes `model`, `view` e `projection`.

A classe `Graphics` é utilizada para armazenar um objeto tridimensional. O método `transform()` prepara a chamada do método `transform()` da classe `Mesh`.

A classe `Transform` é utilizada para calcular a matriz de modelo de um `Object`. O método `translate()` atualiza a posição do objeto. O método `rotate()` atualiza a rotação do objeto. O método `scale()` atualiza o tamanho do objeto. O método `update()` atualiza a matriz do modelo.

A classe `Câmera` é utilizada para armazenar parâmetros da câmera e do volume de visualização e para calcular as matrizes de visualização e de projeção de um `Object`. O método `init()` calcula a matriz de visualização. O método `update()` calcula a matriz de visualização. O método `move()` prepara a chamada dos métodos `translate()`, `rotate()` e `scale()` do `Transform` da `Camera`.

A classe abstrata `Component` é utilizada como base para as classes `Câmera`, `Transform` e `Graphics`.

A classe `Object` é utilizada para armazenar os componentes de uma `Scene`. Em uma cena com duas caixas de madeira, um `Object` será a câmera, outro será a primeira

caixa e outro a segunda caixa. O método `init()` chama os métodos `init()` dos `Component`. O método `update()` chama os métodos `update()` dos `Component`. O método `transform()` chama o método `transform()` do componente `Graphics`.

A classe `Scene` é utilizada para armazenar as informações dos `Object` da cena. O método `init()` chama os métodos `init()` dos `Object`. O método `update()` chama os métodos `update()` dos `Object`. O método `render()` desenha em tela as faces aparentes dentro do volume de visualização.

4.2 Implementação

Esta seção apresenta fragmentos de código do Dream Render, demonstrando aspectos de implementação.

4.2.1 Cálculo das Coordenadas de Tela

A figura 10 descreve o código do método `transform()` da classe `Mesh`, responsável por posicionar um objeto na cena, verificar as faces aparentes dentro do volume de visualização e calcular as coordenadas de tela dos pontos.

```

1  let camera = SceneHandler.activeScene.activeCamera;
2  let transform = this.owner.Transform;
3  let mvp = mat_multMat(mat_multMat(camera.projection, camera.view), transform.model);
4  var renderedFaces = [];
5  for(let i = 0; i < this.faces.length; i++){
6      let p1 = this.faces[i][0];
7      let p2 = this.faces[i][1];
8      let p3 = this.faces[i][2];
9
10     let p1m = mat_multVec(transform.model, this.vertices[p1[0]]);
11     let p2m = mat_multVec(transform.model, this.vertices[p2[0]]);
12     let p3m = mat_multVec(transform.model, this.vertices[p3[0]]);
13
14     let normal = vec_normalize(vec_cross(vec_sub(p2m, p1m), vec_sub(p3m, p1m)));
15     let cameraViewOfTriangle = vec_normalize(vec_sub(p1m, camera.owner.Transform.position))
16
17     (vec_dot(cameraViewOfTriangle, normal) >= 0) && continue;
18
19     p1 = [mat_multVec(mvp, this.vertices[p1[0]]), this.uvs[p1[1]], this.normals[p1[2]]];
20     p2 = [mat_multVec(mvp, this.vertices[p2[0]]), this.uvs[p2[1]], this.normals[p2[2]]];
21     p3 = [mat_multVec(mvp, this.vertices[p3[0]]), this.uvs[p3[1]], this.normals[p3[2]]];
22
23     (p1[0].w <= camera.near || p2[0].w <= camera.near || p3[0].w <= camera.near) && continue;
24     (p1[0].w >= camera.far || p2[0].w >= camera.far || p3[0].w >= camera.far) && continue;
25
26     p1[0] = vec3D(p1[0].x / p1[0].w, p1[0].y / p1[0].w, p1[0].z / p1[0].w, p1[0].w);
27     p2[0] = vec3D(p2[0].x / p2[0].w, p2[0].y / p2[0].w, p2[0].z / p2[0].w, p2[0].w);
28     p3[0] = vec3D(p3[0].x / p3[0].w, p3[0].y / p3[0].w, p3[0].z / p3[0].w, p3[0].w);
29
30     p1[0] = ToScreen(p1[0], Program.canvas);
31     p2[0] = ToScreen(p2[0], Program.canvas);
32     p3[0] = ToScreen(p3[0], Program.canvas);
33
34     renderedFaces.push([p1, p2, p3, extureHandler.getTextura(this.faces[i][3])]);
35 }

```

Figura 10 - Código para o cálculo das coordenadas de tela.

A linha 1 obtém a câmera ativa. A linha 2 obtém a matriz de modelo do objeto. A linha 3 calcula a matriz MVP. Para cada face, são executadas as linhas 6 a 34. As linhas 6 a 8 obtêm os 3 pontos de uma face. As linhas 10 a 12 aplicam a matriz de modelo nos pontos. A linha 14 calcula a normal da face. A linha 15 calcula a direção da câmera até a face. A linha 17 verifica se a face é aparente. As linhas 19 a 21 aplicam a matriz MVP nos pontos. As linhas 23 e 24 verificam se algum ponto da face está dentro do volume de projeção. As linhas 26 a 32 calculam as coordenadas de tela dos pontos. Ao final, `renderedFaces` armazena os dados de todas as faces aparentes dentro do volume de visualização.

4.2.2 Ordenação de Profundidade e Desenho

A figura 11 descreve o código do método `render()` da classe `Scene`, responsável por desenhar as faces aparentes dentro do volume de visualização na tela, ordenadas pela distância da câmera.

```
1  this.imageData = Program.ctx.getImageData(0, 0, Program.canvas.width, Program.canvas.height);
2  let facesRendered = [];
3
4  for(let i = 0; i < this.objectsArr.length; i++){
5      if(this.objectsArr[i].Graphics){
6          facesRendered.push(...this.objectsArr[i].transform());
7      }
8  }
9
10 facesRendered.sort((a,b) =>
11     (a[0].z + a[1].z + a[2].z) / 3 - (b[0].z + b[1].z + b[2].z) / 3
12 );
13
14 for(let i = 0; i < facesRendered.length; i++){
15     if(this.type){
16         this.imageData = fillPaP(...facesRendered[i], this.imageData);
17     } else {
18         this.imageData = fillAfinne(...facesRendered[i], this.imageData);
19     }
20 }
21
22 Program.ctx.putImageData(this.imageData, 0, 0);
```

Figura 11 - Código para a ordenação de profundidade e desenho.

A linha 1 prepara uma imagem em branco do tamanho da tela. As linhas 4 a 6 chamam, para cada `Graphics`, o código descrito na seção 4.2.1. As linhas 10 a 12 ordenam as faces aparentes pela distância da câmera. As linhas 14 a 20 desenharam cada face aparente na imagem de acordo com o método de aplicação de textura da cena. A linha 16 chama o método de aplicação de textura *pixel a pixel*. A linha 18 chama o método de aplicação de textura utilizando transformação afim. Ao final, a linha 22 desenha a imagem na tela.

4.2.3 Desenho Pixel a Pixel

A figura 12 descreve o código da função `fillPaP()`, responsável pela aplicação de textura em uma face utilizando a técnica *pixel a pixel*.

```

1  let tri = [p1, p2, p3];
2
3  let boxIX = Math.min(p1.x, p2.x, p3.x);
4  let boxFX = Math.max(p1.x, p2.x, p3.x);
5  let boxIY = Math.min(p1.y, p2.y, p3.y);
6  let boxFY = Math.max(p1.y, p2.y, p3.y);
7
8  for(let x = boxIX; x < boxFX; x++){
9      for(let y = boxIY; y < boxFY; y++){
10         if(y >= 0 && y < program.canvas.height && x >= 0 && x < program.canvas.width){
11             let weights = baricentric(new vec3D(x, y, 0), tri);
12
13             (weights[0] < 0 || weights[1] < 0 || weights[2] < 0) && continue;
14
15             let color = (texture) ? texture.getPixel(weights, tri) : [255, 255, 255];
16
17             let index = (Math.trunc(y) * image.width + Math.trunc(x)) * 4;
18             image.data[index] = color[0];
19             image.data[index+1] = color[1];
20             image.data[index+2] = color[2];
21         }
22     }
23 }

```

Figura 12 - Código para desenho *pixel a pixel*.

A linha 1 cria uma lista com os três pontos da face. As linhas 3 a 6 calculam o menor e o maior valor de x e de y entre os três pontos. Para cada x e y , entre o menor e o maior valor, são executadas as linhas 10 a 20. A linha 10 verifica se o ponto está dentro da tela. A linha 11 calcula as coordenadas baricêntricas do ponto. A linha 12 verifica se o ponto está dentro da face. A linha 15 obtém o *pixel* na textura utilizando as coordenadas baricêntricas. As linhas 17 a 20 desenharam o *pixel* na imagem.

4.2.4 Desenho com Transformação Afim

A figura 13 descreve o código da função `fillAffine()`, responsável pela aplicação de textura em uma face utilizando transformação afim.

```

1  let x0 = p1[0].x, y0 = p1[0].y;
2  let x1 = p2[0].x, y1 = p2[0].y;
3  let x2 = p3[0].x, y2 = p3[0].y;
4  let u0 = p1[1].x, v0 = p1[1].y;
5  let u1 = p2[1].x, v1 = p2[1].y;
6  let u2 = p3[1].x, v2 = p3[1].y;
7
8  Program.ctx.save();
9  Program.ctx.beginPath();
10 Program.ctx.moveTo(x0, y0);
11 Program.ctx.lineTo(x1, y1);
12 Program.ctx.lineTo(x2, y2);
13 Program.ctx.closePath();
14 Program.ctx.clip();
15
16 let delta = u0*v1 + v0*u2 + u1*v2 - v1*u2 - v0*u1 - u0*v2;
17
18 let delta_a = (x0*v1 + v0*x2 + x1*v2 - v1*x2 - v0*x1 - x0*v2);
19 let delta_b = (u0*x1 + x0*u2 + u1*x2 - x1*u2 - x0*u1 - u0*x2);
20 let delta_c = (u0*v1*x2 + v0*x1*u2 + x0*u1*v2 - x0*v1*u2 - v0*u1*x2 - u0*x1*v2);
21 let delta_d = (y0*v1 + v0*y2 + y1*v2 - v1*y2 - v0*y1 - y0*v2);
22 let delta_e = (u0*y1 + y0*u2 + u1*y2 - y1*u2 - y0*u1 - u0*y2);
23 let delta_f = (u0*v1*y2 + v0*y1*u2 + y0*u1*v2 - y0*v1*u2 - v0*u1*y2 - u0*y1*v2);
24
25 Program.ctx.transform(
26     delta_a / delta, delta_d / delta,
27     delta_b / delta, delta_e / delta,
28     delta_c / delta, delta_f / delta
29 );
30
31 Program.ctx.drawImage(texture, 0, 0);
32 Program.ctx.restore();

```

Figura 13 - Código para desenho com transformação afim.

As linhas de 1 a 3 renomeiam as coordenadas dos três pontos da face. As linhas 4 a 6 renomeiam as coordenadas de textura dos três pontos da face. As linhas 9 a 14 desenharam um triângulo na tela demarcando uma área de corte (*clipping*). As linhas 16 a 23 calculam os valores a , b , c , d , e e f da matriz M_{Affine} descrita na seção 2.3.1. Ao final, nas linhas 25 a 29 é chamado o método `transform()` da Canvas API com os valores a , b , c , d , e e f calculados e na linha 31, desenhada a textura na tela.

5. RESULTADOS

Este capítulo apresenta os resultados obtidos pelo motor gráfico Dream Render. Iniciando com capturas de tela e terminando com um comparativo de desempenho entre vários dispositivos e métodos de aplicação de textura.

5.1 Capturas de Tela

As figuras 14 e 15 apresentam as capturas de tela do Dream Render.

A figura 14 mostra a tela com o menu de cenas na esquerda, com seis cenas carregadas: **Box_PAP**, **Houses_PAP**, **Man_PAP**, **Box_Affine**, **Houses_Affine** e **Man_Affine**. A cena sendo desenhada é a **Houses_PAP** que possui quatro objetos: **house_01**, **house_02**, **house_03** e **Camera_01**. O botão **Upload File** no canto superior direito permite escolher uma cena para carregar.



Figura 14 - Tela com menu.

A figura 15 mostra a tela de navegação com os controles analógicos nos cantos inferiores. O controle do canto inferior esquerdo move a câmera e o do canto inferior direito, rotaciona a câmera.



Figura 15 - Tela de navegação.

5.2 Comparativo de Desempenho

Para o comparativo de desempenho foram utilizadas três cenas com diferentes quantidades de faces triangulares:

Cena 1: uma caixa, com 12 faces.

Cena 2: três casas, com 2574 faces.

Cena 3: um homem, com 48918 faces.

Foram utilizados os seguintes dispositivos, três móveis e um computador:

Dispositivo 1: Samsung Galaxy A04S, processador Exynos 850 (Octa-core 2.0 GHz Cortex-A55), resolução 720 x 1600 *pixels*, 3GB de memória RAM, navegador Google Chrome.

Dispositivo 2: Motorola Moto G73, processador Dimensity 930 (2x 2.2 GHz Cortex-A78 + 6x 2.0 GHz Cortex-A55), resolução 1080 x 2400 *pixels*, 8GB de memória RAM, navegador Google Chrome.

Dispositivo 3: iPhone 12, processador Apple A14 Bionic (2x 3.1 GHz Firestorm + 4x 1.8 GHz Icestorm), resolução 1170 x 2532 *pixels*, 4GB de memória RAM, navegador Google Chrome.

Dispositivo 4: ASUS VivoBook 15 X540UAR, processador Intel Core i5-6200U (2 núcleos, 4 threads, de 2.30GHz a 2.80GHz), resolução 1366 x 768 *pixels*, 8GB de memória RAM, navegador Firefox.

E foram utilizados dois métodos de aplicação de textura:

Método 1: *pixel a pixel*.

Método 2: transformação afim.

O desempenho foi medido em quadros (*frames*) desenhados por segundo (FPS) utilizando cada método de aplicação de textura em cada dispositivo em cada cena. Um *frame* é o resultado de todo o processo, desde o cálculo das coordenadas de tela da seção 4.2.1, ordenação de profundidade e desenho da seção 4.2.2 com aplicação de textura pixel a pixel da seção 4.2.3 e transformação afim da seção 4.2.4.

As tabelas 1 a 4 detalham o FPS mínimo e máximo obtido para cada combinação de cena, dispositivo e método de aplicação de textura. O tempo mínimo foi obtido quando todos os objetos na cena estavam dentro do volume de visão tão próximos quanto possível da câmera. O tempo máximo foi obtido quando todos os objetos na cena estavam a mais de 300 de distância da câmera.

Tabela 1 - Desempenho do dispositivo Samsung Galaxy A04S.

	Pixel a pixel	Transformação afim
12 faces	6 a 37 FPS	60 a 60 FPS
2574 faces	2 a 6 FPS	7 a 9 FPS
48918 faces	1 a 1 FPS	1 a 2 FPS

Tabela 2 - Desempenho do dispositivo Motorola Moto G73.

	Pixel a pixel	Transformação afim
12 faces	35 a 89 FPS	90 a 90 FPS
2574 faces	5 a 17 FPS	20 a 22 FPS
48918 faces	2 a 9 FPS	11 a 11 FPS

Tabela 3 - Desempenho do dispositivo iPhone 12.

	Pixel a pixel	Transformação afim
12 faces	57 a 60 FPS	60 a 60 FPS
2574 faces	4 a 19 FPS	21 a 23 FPS
48918 faces	3 a 11 FPS	13 a 13 FPS

Tabela 4 - Desempenho do dispositivo ASUS VivoBook 15.

	Pixel a pixel	Transformação afim
12 faces	24 a 60 FPS	60 a 60 FPS
2574 faces	4 a 13 FPS	17 a 20 FPS
48918 faces	1 a 6 FPS	7 a 11 FPS

Duas conclusões podem ser observadas nas tabelas 1 a 4. Primeiro, quanto ao impacto do método de aplicação de textura utilizado. Segundo, quanto ao impacto do processador utilizado.

5.2.1 Impacto do Método de Aplicação de Textura

Analisando os tempos máximos obtidos desenhando a cena com 48918 faces nos dispositivos, o método de aplicação de textura por transformação afim foi 55,25% mais rápido em média que o por *pixel a pixel*.

Entretanto, como pode ser observado na figura 16, o método de aplicação de textura por transformação afim, apresenta defeito na sensação de perspectiva e o por *pixel a pixel*, não. Na figura 16, a imagem da esquerda é a textura original sem perspectiva, a ao centro é a textura em perspectiva resultante do método por

transformação afim e a da direita é a textura em perspectiva resultante do método *pixel a pixel*.

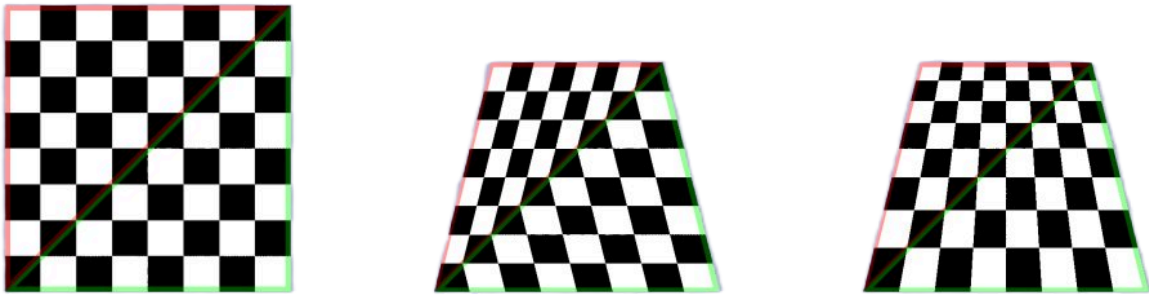


Figura 16 - Diferença na sensação de perspectiva entre os métodos de aplicação de textura.

Fonte: https://en.wikipedia.org/wiki/Texture_mapping

5.2.2 Impacto do Processador

Também analisando os tempos máximos obtidos desenhando a cena com 48918 faces nos dispositivos apenas com o método de aplicação de textura *pixel a pixel*, pode ser percebido que, quanto maior a frequência do processador utilizado, maior o FPS obtido. Realizando um ajuste linear nos dados obtidos, foi calculado uma tendência $FPS = 5,7460GHz - 7,7587$, com um coeficiente de determinação de 45,82%. Isto significa dizer que 1GHz a mais no processador, resultará em aproximadamente 6 FPS a mais no desempenho do motor gráfico.

6. CONSIDERAÇÕES FINAIS

Este projeto surgiu como um experimento sobre computação gráfica. Durante o desenvolvimento foram estudados os fundamentos matemáticos da computação gráfica e foi implementado um motor gráfico capaz de desenhar cenas tridimensionais com aplicação de textura. Foram realizados testes com dois métodos diferentes de aplicação de textura em diversos dispositivos e se determinou que, embora seja possível realizar o desenho de cenas tridimensionais apenas utilizando a CPU, isto não é aconselhável devido ao custo computacional. Nestes testes, foi evidenciado que o método de aplicação de textura por transformação afim é mais rápido mas apresenta defeito na sensação de perspectiva e que a quantidade de quadros desenhados por segundo é proporcional à frequência do processador utilizado.

Como sugestão para trabalhos futuros se aponta a otimização do código do motor gráfico, a adição de outros métodos de aplicação de textura, a adição de iluminação e seus efeitos e adição de métodos de colisão entre objetos. Além disso, a organização das classes permite que novos componentes sejam criados, possibilitando, por exemplo, a implementação de lógicas de movimentação de objetos pela cena.

Foi um prazer desenvolver o Dream Render e se espera que este projeto incentive outros a aprender sobre computação gráfica e criar seus próprios projetos. Para os interessados, o código fonte do Dream Render está disponível em <https://github.com/ThiagoOSA/Dream-Render>.

7. REFERÊNCIAS

BLUNDELL, Barry G. **An introduction to computer graphics and creative 3-D environments**. [s.l.] Springer Science & Business Media, 2008.

DE VRIES, Joey. **Learn OpenGL-Graphics Programming: Learn Modern OpenGL Graphics Programming in a Step-by-step Fashion**. [s.l.] Kendall & Welling, 2020.

IEZZI, G.; DOLCE, O.; Degenszajn, D.; PÉRIGO, R.; ALMEIDA, N. **Matemática: Ciência e Aplicações - Ensino Médio**. Volume 2. 9ª edição. [s.l.] Saraiva, 2016.

MOZILLA DEVELOPER NETWORK. **HTML: Linguagem de Marcação de Hipertexto**. Disponível em <https://developer.mozilla.org/pt-BR/docs/Web/HTML>, acessado em 26/06/2025.

MOZILLA DEVELOPER NETWORK. **JavaScript**. Disponível em <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript>, acessado em 26/06/2025.

MOZILLA DEVELOPER NETWORK. **Canvas - APIs da Web**. Disponível em https://developer.mozilla.org/pt-BR/docs/Web/API/Canvas_API, acessado em 28/10/2025.

MURRAY, James D; VAN RYPER, William. **Encyclopedia of Graphics File Formats**. [s.l.] O'Reilly Media, 1996.

SALOMON, David. **Transformations and projections in computer graphics**. [s.l.] Springer, 2006.

GLOSSÁRIO

Clipping Em computação gráfica, *clipping* é um método para habilitar ou desabilitar seletivamente operações de renderização dentro de uma região de interesse definida.

Frame *Frame*, também conhecido como quadro de vídeo, é cada uma das imagens fixas de um produto audiovisual, que quando apresentadas rapidamente em sequência criam a ilusão de movimento.

Framework Em desenvolvimento de *software*, o *framework* é uma estrutura guia com padrões e ferramentas reusáveis para desenvolver programas computacionais de forma rápida.

Pixel Pixel é o menor elemento em um dispositivo de exibição (por exemplo, um monitor), ao qual é possível atribuir-se uma cor.

Software *Software* é uma coleção de programas que dizem a um computador como executar tarefas específicas.

Web *World Wide Web (web)* é um sistema de informação que permite o compartilhamento de conteúdo pela Internet usando uma interface gráfica de usuário .